Python level 2 – les 5: app die QR code aanmaakt

1. Welke modules gaan we nodig hebben?
    a. Windows app ➔ tkinter
    b. Grafisch ➔ Image en ImageTk uit library PIL (package “Pillow”)
    c. API aanspreken ➔ requests
    d. Communicatie met operating system ➔ io

```
import tkinter
from PIL import Image, ImageTk
import requests
import io
```

!! Om “PIL” te kunnen gebruiken: eerst in terminal commando uitvoeren: pip install Pillow

2. We maken een windows-programma met behulp van framework tKinter, de handle van onze applicatie houden we bij in variabele “**venster**”

```
venster = tkinter.Tk()
venster.mainloop()
```

3. We maken een klasse “**QrGeneratorApp**”.
    a. In de constructor van die klasse wordt de instance die dan gemaakt wordt (**self**) gekoppeld aan een window dat we als parameter “**venster**” meegeven.
    b. In de main-code van ons programma gaan we de constructor van de klasse aanroepen om één instance van deze klasse te maken.
        i. We houden die nieuw aangemaakte instance bij in variabele “**app**”
        ii. We geven het window (variabele “**venster**”) dat we in stap 2 gemaakt hebben mee als parameter

```
class QrCodeGeneratorApp:
    new *
    def __init__(self, venster):
        self.venster = venster


venster = tkinter.Tk()
app = QrCodeGeneratorApp(venster)
venster.mainloop()
```

Python level 2 – les 5: app die QR code aanmaakt

4. In de constructor van de klasse zetten we nu het windows formulier helemaal klaar
    a. Venster goed zetten
        i. Afmetingen venster (geometry): 600 bij 400 pixels
        ii. Titel kiezen (vrij te kiezen tekst)
    b. Label maken met “voer de URL in : “ als tekst.
    c. Invoerveld genaamd “**url_entry**” waar de gebruiker een URL kan in plakken
    d. Een knop genaamd “**generate_button**” maken, met als tekst “Genereer QR-code” en als commando een aanroep van de klasse-functie “**genereer_qr_code**” (deze functie bestaat nog niet, die maken we in stap 5)
    e. Een label genaamd “**qr_code_image_label**”

```
def __init__(self, venster):
    self.venster = venster
    self.venster.geometry('600x400+200+200')
    self.venster.title("QR-code Generator")

    self.label = tkinter.Label(venster, text="Voer de URL in:")
    self.label.pack(pady=10)

    self.url_entry = tkinter.Entry(venster)
    self.url_entry.pack()

    self.generate_button = tkinter.Button(venster,
                text="Genereer QR-code",
                command=self.genereer_qr_code)
    self.generate_button.pack(pady=10)

    self.qr_code_image_label = tkinter.Label(venster)
    self.qr_code_image_label.pack()
```

5. Eigen functie voor de klasse maken genaamd “**genereer_qr_code**”. Hierin gaat het echte werk gebeuren

5.1 we beginnen alvast met de inhoud van het invoerveld “**url_entry**” in variabele “**url**” te stoppen

```
def genereer_qr_code(self):
    url = self.url_entry.get()
```

Python level 2 – les 5: app die QR code aanmaakt

5.2 als de url goed is (niet leeg is) doen we een aanroep naar de API van online service “qrserver.com”.

1. maak de correcte URL voor de request, en houd die bij in variabele “**qr_code_url**”
2. doe een get-request en houd het resultaat bij in variabele “**response**”

```
if url:
    qr_code_url = f"https://api.qrserver.com/v1/create-qr-code/?data={url}&size=200x200"
    response = requests.get(qr_code_url)
```

5.3 als de request naar de API goed gegaan is (status_code gelijk aan 200) dan gaan we het antwoord verwerken

1. inhoud van de response bijhouden in variabele “**afbeelding_data**”
2. hieruit gaan we de afbeelding halen
    - 2.1 maak een input/output in-memory bytestream met **afbeelding_data** als parameter
    - 2.2 gebruik het resultaat hiervan om een Image te openen
    - 2.3 stop het resultaat hiervan in variabele “**qr_code_afbeelding”**

```
if response.status_code == 200:
    afbeelding_data = response.content
    qr_code_afbeelding = Image.open(io.BytesIO(afbeelding_data))
```

3. De afbeelding is nu uit de response gefilterd, maar bestaat alleen nog maar in het geheugen van de computer. Nu gaan we de afbeelding tonen op het scherm
    - 3.1 transformeer de afbeelding naar een tKinter PhotoImage,

      dus iets dat tKinter op het scherm kan zetten als afbeelding.

      en sla op in variabele “**afbeelding**”
    - 3.2 nu moeten we die afbeelding nog in het label krijgen (self.qr_code_image_label)
        - 3.2.1 ken in de config van het label de afbeelding toe aan eigenschap “**image**”
        - 3.2.2 ken de afbeelding ook toe aan property “**image**” van het label

```
3.1   afbeelding = ImageTk.PhotoImage(qr_code_afbeelding)
3.2.1 self.qr_code_image_label.config(image=afbeelding)
3.2.2 self.qr_code_image_label.image = afbeelding
```

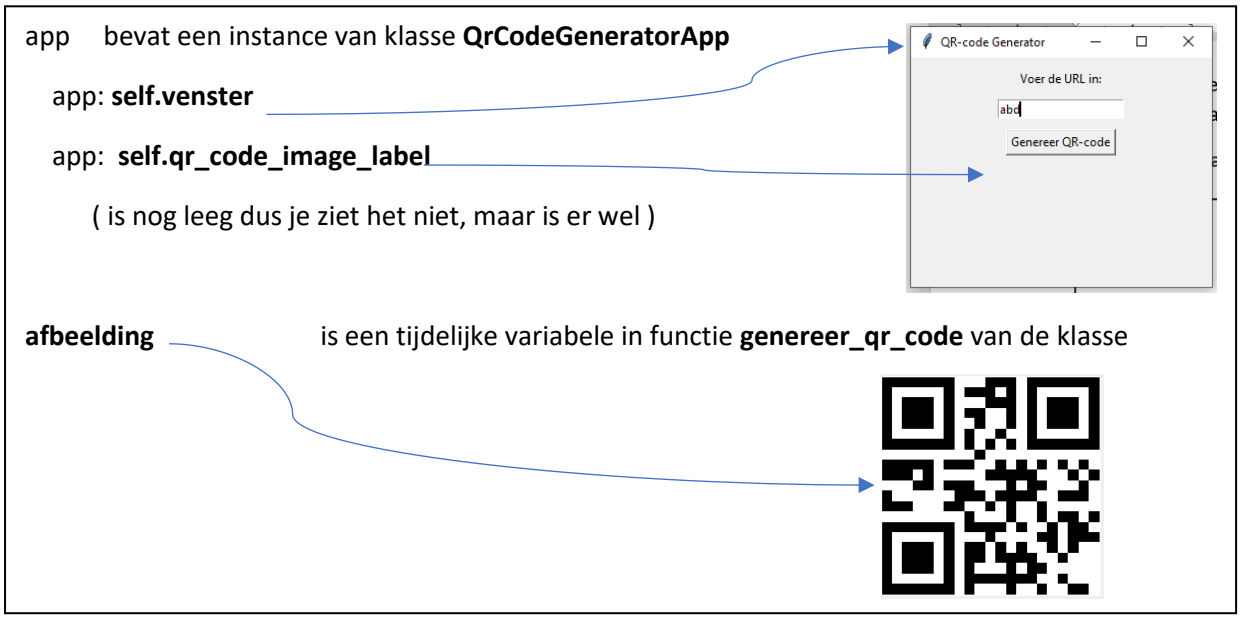
Python level 2 – les 5: app die QR code aanmaakt

Nog een beetje uitleg

Waarom die variabele “afbeelding” twee keer toekennen aan hetzelfde label, één keer in de config en één keer met de property?

Dit heeft te maken met een iets moeilijker concept uit de computerwetenschappen, “**garbage collection**”. Het komt erop neer dat de computer, of meer bepaald de python interpreter die je programma draait, de hele tijd probeert om zo efficiënt mogelijk te werken, en zo weinig mogelijk geheugen te gebruiken. Garbage collection is daar een onderdeel van: variabelen die “buiten scope gaan” worden weggegooid, en geheugenplaatsen die door geen enkele variabele meer gereferenced worden, worden weer vrijgegeven. Zo houdt de interpreter zijn omgeving, de space waarin het programma wordt uitgevoerd, zo proper mogelijk.

Na uitvoeren van stap 3.1 kan je het geheugen voor je applicatie schematisch zo zien:



Op dit moment is de **reference-count** van de geheugenplaats waar het prentje staat gelijk aan 1: er wordt naar verwezen in variabele “**afbeelding**”.

Door in stap 3.2.1 de inhoud van variabele “**afbeelding**” toe kennen aan de config-eigenschap “image” van het label, zeg je eigenlijk tegen het label: “je moet een prentje gaan tonen op het scherm, en de gegevens van dat prentje vind je op de geheugenplaats waar variabele “afbeelding” naar verwijst.

De applicatie zal dan intern bij het label de locatie van de te tonen afbeelding-inhoud zetten op dezelfde geheugenplaats als variabele “**afbeelding**”.

Dus de image-pointer van het label gaat naar dezelfde geheugenplaats wijzen als de variabele “afbeelding”.

Het probleem is, dat tKinter objecten hun configuratie-informatie niet echt als variabelen opslaan, maar een intern formaat, dat de garbage collector niet herkent. Dit is namelijk **rendering-informatie**, die vertelt wat de grafische eigenschappen van alle elementen zijn: hoe hoog, hoe breed, welke kleur, welk lettertype, waar het moet staan, welk prentje het moet tonen, …

Python level 2 – les 5: app die QR code aanmaakt

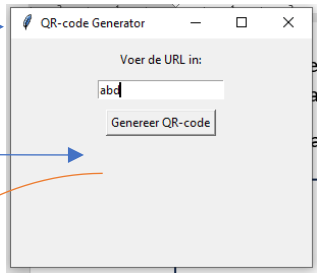
Na stap 3.2.1 ziet de geheugen-structuur van je app er schematisch zo uit:

app bevat een instance van klasse **QrCodeGeneratorApp**

app: **self.venster**

app: **self.qr_code_image_label**

(is nog leeg dus je ziet het niet, maar is er wel)



afbeelding is een tijdelijke variabele in functie **genereer_qr_code** van de klasse

Het enige wat er bij gekomen is, is de rode pijl. Die rode pijl betekent dat het label "weet" dat het dit prentje moet tonen.

Dus de reference-count van de geheugenplaats van het prentje is nog steeds 1, is niet verhoogd: alleen de variabele "**afbeelding**" verwijst ernaar. Het label 'weet' ook dat ie z'n grafische data hier moet gaan halen, maar dat telt niet, dit wordt niet als een echte reference gezien.

Wat gebeurt er nu als we stap 3.2.2 niet zouden uitvoeren?

Na uitvoering van de code in het if-blok (**if response.status_code == 200**) gaat variabele "**afbeelding**" buiten scope. Die bestaat alleen zolang we in dat if-blok zitten met de uitvoering. Deze wordt dus opgeruimd.

En als variabele "**afbeelding**" weg is, dan is er ook niks of niemand meer dat verwijst naar die grote lap grafische data, dus die kan ook weg. De reference-count van de geheugenplaats met het prentje is nul geworden omdat variabele "**afbeelding**" niet meer bestaat.

Hierna staat de image-pointer van het label dus te wijzen naar niks, en kan er dus ook geen prentje getoond worden. Dat zou er zo uit zien:

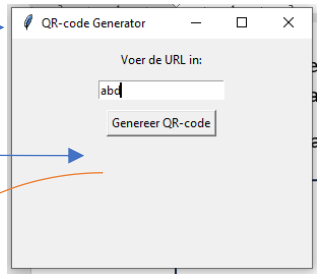
app bevat een instance van klasse **QrCodeGeneratorApp**

app: **self.venster**

app: **self.qr_code_image_label**

(is nog leeg dus je ziet het niet, maar is er wel)

"**afbeelding**" is weg, opgekuist omdat ie uit scope is gegaan



?????

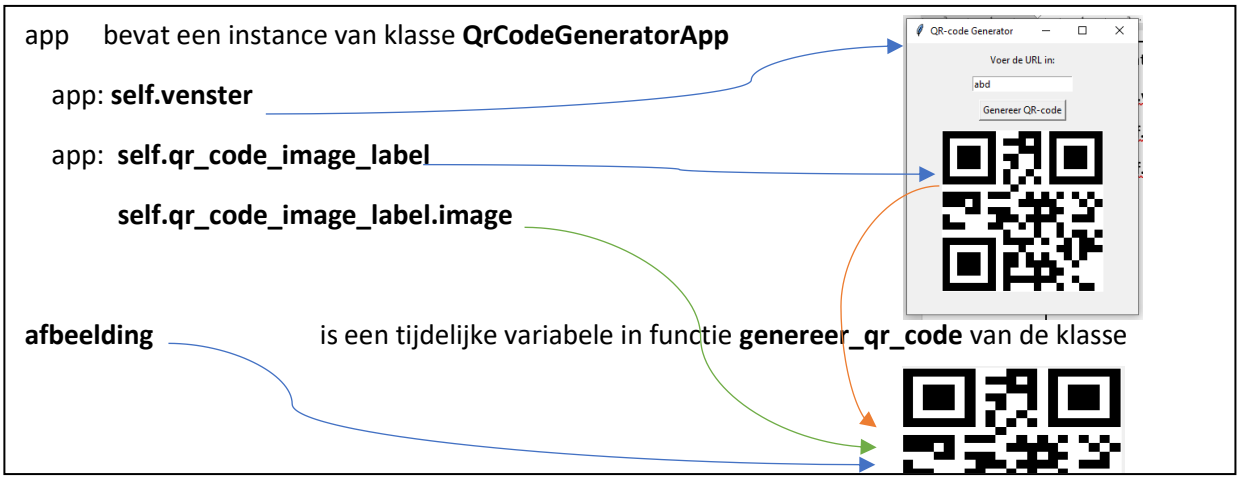
De blok grafische data waar "**afbeelding**" naar verwees is ook weg, de reference count hiervan is nul geworden omdat de rode pijl config-informatie is en geen echte referentie.

m.a.w. de rode pijl is niet genoeg om de garbage collector tegen te houden bij het weggooien van het prentje

Python level 2 – les 5: app die QR code aanmaakt

We moeten dus een manier vinden om de interpreter wijs te maken dat hij de geheugenplaats met het prentje niet mag garbage-collecten. Dat doen we in stap 3.2.2 door aan te geven dat het label in self.qr_code_image_label ook nog een property "**image**" heeft, die OOK verwijst naar de inhoud van variabele "**afbeelding**".

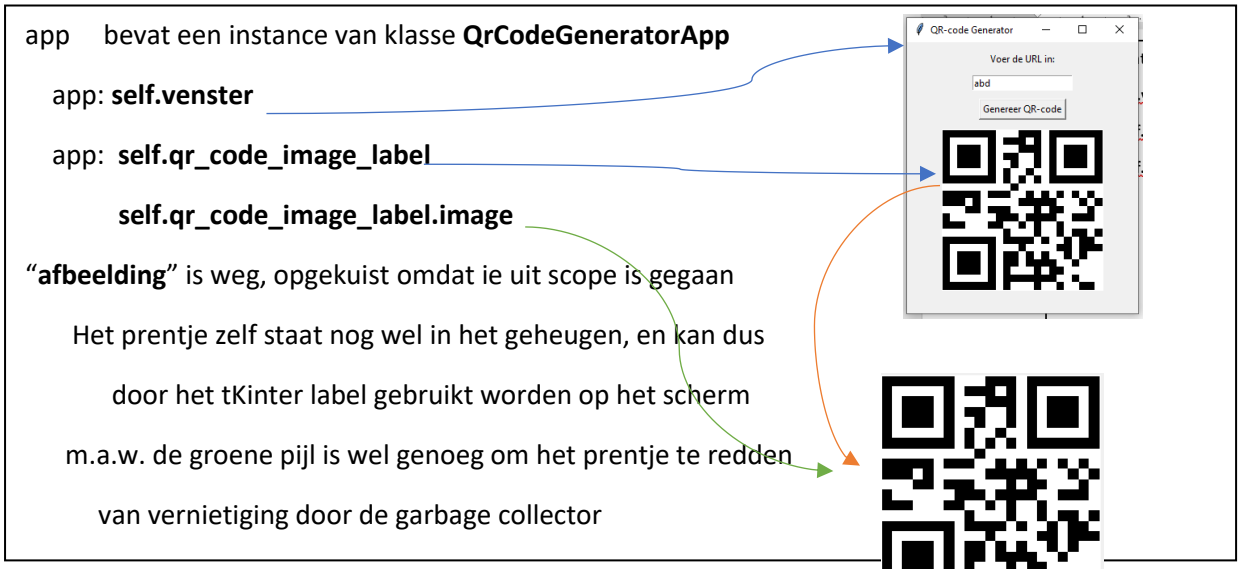
En een property wordt WEL behandeld als een echte variabele. Na uitvoering van stap 3.2.2 ziet het geheugen er schematisch zo uit:



Je ziet het goed, de groene pijl komt erbij, alweer een pointer, de derde al, die naar hetzelfde stukje geheugen verwijst. De reference-count van de geheugenplaats met het prentje staat nu op 2: één van variabele "**afbeelding**" en één van de property "**image**" van het label. De rode pijl van config-informatie van het label telt nog steeds niet mee.

Als nu de garbage collector aan de slag gaat, zal hij opnieuw correct vaststellen dat variabele "**afbeelding**" zijn rol gespeeld heeft, buiten scope is gegaan, en die weggooien.

Als hij daarna gaat checken of dan ook de lap grafische data weg mag waar variabele "**afbeelding**" naar verwijst, dan zal hij vaststellen dat dit **NIET** het geval is, omdat nu de property "**image**" van het label hier nog naar verwijst, met andere woorden de reference-count van dit stukje geheugen is nog steeds groter dan nul. (namelijk = 1). De eind-toestand in het geheugen is dus deze:



Conclusie: stappen 3.2.1 en 3.2.2 zijn dus allebei nodig.

De eerste om het label te vertellen waar het z'n grafische data moet halen (rode pijl).

De tweede om die grafische data te beschermen tegen de garbage collector (groene pijl).